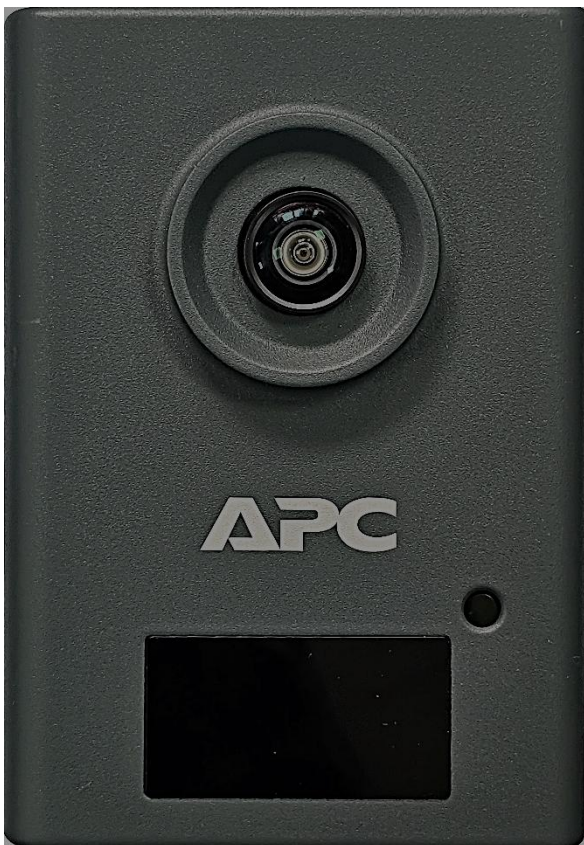


KAVRO

REST API User Manual



Version 1.2

e-con Systems®

01 - September - 2026



e-con Systems®

Think Camera. Think e-con.

Contents

INTRODUCTION TO KAVRO	6
DEVICE INFO	7
DEVICE STATUS INFO	9
FACTORY RESET API	11
REBOOT API	13
DOWNLOAD LOG API	15
CLEAR LOG API	17
UPDATE ADMIN PASSWORD API	19
ADD USER API	21
UPDATE USER API	23
REMOVE USER API	25
GET HTTP MODE API	27
SET HTTP MODE API	29
GET IPV4 CONFIG API	31
SET IPV4 CONFIG API	33
GET IPV6 CONFIG API	35
GET ONVIF STATUS API	37
SET ONVIF STATUS API	39
GET RTP STATUS API	41
SET RTP STATUS API	43

<u>GET RTP SESSIONS API</u>	<u>45</u>
<u>TERMINATE RTP SESSIONS API</u>	<u>47</u>
<u>GET WEB UI STATUS API</u>	<u>49</u>
<u>SET WEB UI STATUS API</u>	<u>51</u>
<u>GET RSTP CONFIGURATION API</u>	<u>53</u>
<u>SET RSTP CONFIGURATION API</u>	<u>55</u>
<u>GET SYSTEM TIME API</u>	<u>57</u>
<u>SET SYSTEM TIME API</u>	<u>59</u>
<u>GET TIME SETTINGS API</u>	<u>61</u>
<u>UPDATE TIME SETTINGS API</u>	<u>63</u>
<u>DOWNLOAD CERTIFICATE API</u>	<u>65</u>
<u>DOWNLOAD CERTIFICATE API</u>	<u>67</u>
<u>UPLOAD CERTIFICATE API</u>	<u>69</u>
<u>GENERATE SELF-SIGNED CERTIFICATE API</u>	<u>71</u>
<u>RESTORE DEFAULT CERTIFICATE API</u>	<u>73</u>
<u>GET COMMON VIDEO SETTINGS API</u>	<u>75</u>
<u>SET COMMON VIDEO SETTINGS API</u>	<u>77</u>
<u>GET VIDEO STREAMS API</u>	<u>79</u>
<u>GET SPECIFIC VIDEO STREAM API</u>	<u>82</u>
<u>SET SPECIFIC VIDEO STREAM API</u>	<u>84</u>
<u>GET IMAGE SETTINGS API</u>	<u>88</u>

<u>SET IMAGE SETTINGS API</u>	<u>90</u>
<u>RESTORE IMAGE DEFAULTS API</u>	<u>94</u>
<u>GET ROI EXPOSURE</u>	<u>96</u>
<u>SET ROI EXPOSURE</u>	<u>98</u>
<u>GET IMAGE FRAME API</u>	<u>100</u>
<u>GET OVERLAY API</u>	<u>102</u>
<u>SET OVERLAY API</u>	<u>104</u>
<u>GET MOTION CONFIGURATION API</u>	<u>106</u>
<u>SET MOTION CONFIGURATION API</u>	<u>108</u>
<u>GET MOTION EVENTS API</u>	<u>110</u>
<u>CLEAR MOTION EVENTS API</u>	<u>112</u>
<u>GET MOTION STATUS API</u>	<u>114</u>
<u>GENERATE VIDEO CLIP API</u>	<u>116</u>
<u>GET CLIP STATUS API</u>	<u>118</u>
<u>DOWNLOAD VIDEO CLIP API</u>	<u>120</u>
<u>GET SD CARD STATUS API</u>	<u>122</u>
<u>GET SD CARD USAGE API</u>	<u>124</u>
<u>FORMAT SD CARD API</u>	<u>126</u>
<u>RECOVER SD CARD API</u>	<u>128</u>
<u>MOUNT SD CARD API</u>	<u>130</u>
<u>UNMOUNT SD CARD API</u>	<u>132</u>

<u>GET FIRMWARE VERSION API</u>	<u>134</u>
<u>FIRMWARE UPLOAD API</u>	<u>136</u>
<u>FIRMWARE UPDATE API</u>	<u>138</u>
<u>GET FIRMWARE UPDATE STATUS API</u>	<u>140</u>
<u>RELATED DOCUMENTS</u>	<u>142</u>
<u>GLOSSARY</u>	<u>143</u>
<u>REVISION HISTORY</u>	<u>144</u>

Introduction to KAVRO

Schneider Electric, in partnership with e-con Systems, is developing and manufacturing the next-generation Camera Pod 166. As part of this collaboration, e-con Systems has designed a customized IP camera, designated RouteCAM_P_CU514_CHLW_BX_H03R1, specifically for Schneider Electric. The RouteCAM_P_CU514_CHLW_BX_H03R1 camera is intended for surveillance applications within server rooms, enabling reliable security monitoring and footage capture in environments with limited ambient lighting. The camera is built around Sony's IMX335 5-megapixel CMOS image sensor, featuring a pixel size of $2\text{ }\mu\text{m} \times 2\text{ }\mu\text{m}$, to deliver high-quality image capture. To support operation in low-light and dark conditions, the camera incorporates integrated infrared (IR) illumination and provides automatic day/night mode switching based on readings from an onboard ambient light sensor. The device also includes a configurable motion detection feature and continuously records and stores 1-second video clips on a microSD card. For installation flexibility, the camera supports Power over Ethernet (PoE), enabling both power and data transmission over a single Ethernet cable. Additionally, it supports daisy-chain connectivity of up to 4 cameras within a single chain. An auxiliary DC power input jack is also provided as an alternative power source. The RouteCAM_P_CU514_CHLW_BX_H03R1 camera supports network discovery and interoperability through the ONVIF standard. e-con Systems has further customized the ONVIF implementation to ensure seamless integration with Schneider Electric's existing software ecosystem. To enhance device maintainability and security, the camera supports Over-the-Air (OTA) firmware updates and incorporates Secure Boot functionality.

Device Info

This section describes the API for retrieving static hardware and identity details of the device such as firmware version, mac address, serial number.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API to verify if the get the device info has been received successfully from the camera.

Endpoint: `/device/info`

Method: `GET`

Basic Authentication:

Username:xxx

Password:Test@123

Request Body (JSON) :

```
{  
}
```

Example URL

https://<ipaddress-FastCGI server>/api/v1/device/info

Responses

Status Code: 200 (Ok) – This status code from the camera indicates the device info was retrieved successfully.

Response Body:

```
{  
  "deviceType": "CameraPod",  
  "model": "NB166",  
  "firmwareVersion": "1.141.3",  
  "serialNumber": "123456789",  
}
```

```
"macAddress": "02:00:06:00:10:06",  
"dhcp_mode": "DHCP",  
"ipv4_address": "192.168.0.3",  
"ipv6_address_scope_link": "fe80::6ff:fe00:1006"  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The below API will be received in response to the request from the camera.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section explains how the device info REST API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.

Device Status Info

This section describes the API for retrieving real-time operational metrics. Unlike static device information, this endpoint provides dynamic data regarding the system's health, and resource consumption.

Base URL

`https://<ipaddress-FastCGI server>/api/v1`

Request

Run the following API to retrieve the runtime information of the camera.

Endpoint: ``/device/status``

Method: ``GET``

Basic Authentication:

Username: `xxx`

Password: `Test@123`

Request Body (JSON) :

```
{  
}
```

Example URL

`https://<ipaddress-FastCGI server>/api/v1/device/status`

Responses

Status Code: 200 (Ok) – This status code from the camera indicates that the time information has been successfully retrieved.

Response Body:

```
{  
  "uptimeSeconds": 1353,  
  "cpuLoad": 1.63,  
  "memoryUsedBytes": 618541056,  
  "temperatureC": 51
```

```
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The below API will be received in response to the request from the camera.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section explains how the device status REST API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.

Factory Reset API

This section describes the API for performing a factory reset, which reverts the camera to factory-default state.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on the postman to perform a factory reset on the device.

```
Endpoint: `/device/factory-reset`  
Method: `POST`  
Basic Authentication:  
Username:xxx  
Password:Test@123  
  
Request Body (JSON) :  
{  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/device/factory-reset.

Responses

Status Code: 200 (Ok) – This status code from the camera indicates that the factory reset has been successfully completed.

Response Body:

```
{  
  "status": "accepted"  
}
```

Note: After successful, the device will be restarted.

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how the factory reset API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.

Reboot API

This section describes the API for performing a reboot of the camera without doing factory reset.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on the postman to perform a reboot on the device.

```
Endpoint: `/device/reboot`  
Method: `POST`  
Basic Authentication:  
Username:xxx  
Password:Test@123  
  
Request Body (JSON) :  
{  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/device/reboot

Responses

Status Code: 200 (Ok) – This status code from the camera indicates that the reboot has been successfully completed.

Response Body:

```
{  
  "status": "accepted"  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how the reboot API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.

Download Log API

This section details the API for downloading critical logs essential for diagnosing issues. These logs provide vital information for troubleshooting and system analysis.

Note: Timestamps in the downloaded log file reflect the device clock as last configured through the Set System time API (`/time/clock`); changing the system time changes the timestamps written to subsequent log entries. Log lines generated by the streamer process (streamer2) additionally honour the time zone offset and Daylight Saving Time settings configured through the Update Time Settings API (`/time/settings`), so their timestamps may appear different from other log entries, which are recorded in UTC.

Base URL

`https://<ipaddress-FastCGI server>/api/v1`

Request

Run the following API on the postman to download all log from the device.

```
Endpoint: `/device/logs`  
Method: `GET`  
  
Basic Authentication:  
Username: xxx  
Password: Test@123  
  
Request Body (JSON):  
{  
}
```

Example

`https://<ip-address- fastCGI server>/api/v1/device/logs`

Note:

Instead of clicking the “Send” button, click the down arrow on the right side of the “Send” button, and then select “Send and Download.”

Responses

Status Code: 200 (Ok) – This status code from the camera indicates that the “logs.txt” file has been successfully downloaded.

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how the download log API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.

Clear Log API

This section details the API for removing logs stored on the camera.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on the postman to clear all the logs from the camera.

Endpoint: ``/device/logs/clear``

Method: ``POST``

Basic Authentication:

Username: `xxx`

Password: `Test@123`

Request Body (JSON) :

```
{  
  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/device/logs/clear

Responses

Status Code: 200 (Ok) – This status code from the camera indicates that the logs file has been successfully cleared.

Request Body:

```
{  
  
  "status": "accepted"  
  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how the clear log API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.

Update Admin Password API

This section describes the API for updating the administrator's password, allowing an authenticated admin user to securely change their current password.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API to verify if the admin password has been successfully changed for the camera.

```
Endpoint: `/users/default/password/change`  
Method: `POST`  
  
Basic Authentication:  
Username:xxx  
Password:Test@123  
  
Request Body (JSON):  
{  
  "newPassword": "Test@12345"  
}
```

Example URL

https://<ipaddress-FastCGI server>/api/v1/users/default/password/change

Responses

Status Code: 200 (Ok) – This status code from the camera indicates the admin password change status of the camera.

Response Body:

```
{  
  "status": "updated"  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The below API will be received in response to the request from the camera.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section explains how the admin password API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.

Add User API

This section describes the AddUser API, which allows administrators to create new user accounts on the platform. This API enables the addition of user credentials, such as username, password, and user roles (e.g., admin, operator, viewer). Roles define the user's access level and permissions for interacting with cameras and related resources. This API facilitates secure, role-based access control, ensuring that only authorized users can access specific functions within the platform.

Note: Access to the AddUser API is restricted to admin users only.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on the Postman to add user in the table.

```
Endpoint: `/users/add`  
Method: `POST`  
  
Basic Authentication:  
Username: xxx  
Password: Test@123  
  
Request Path Parameters (JSON) :  
{  
  "username": "test",  
  "password": "test@1234567",  
  "permissions": {  
    "camera_control": true,  
    "camera_network": true,  
    "log_access": false  
  }  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/users/add

Responses

Status Code: 200 (Ok) – This status code from the camera indicates that the user has been added successfully.

Response Body:

The below API will be received in response to the request from the host PC.

```
{  
  "success": true,  
  "message": "User added successfully"  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The below API will be received in response to the request from the camera.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section explains how the add user API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.

Update User API

This section describes the Update User API, which allows administrators to modify existing user account details, such as username, password, and role-based permissions (e.g., updating a user's role from viewer to operator). This API ensures that user roles and access levels can be managed dynamically, providing flexibility in adjusting permissions as needed.

Note: Access to the Update User API is restricted to admin users only.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on the postman to update user details or status after camera changes have been applied.

Endpoint: ``/users/update``

Method: ``POST``

Basic Authentication:

Username: `xxx`

Password: `Test@123`

Request Path Parameters (JSON) :

```
{
  "username": "test",
  "password": "test@1234567",
  "permissions": {
    "camera_control": false,
    "camera_network": false,
    "log_access": true
  }
}
```

Example

https://<ipaddress-FastCGI server>/api/v1/users/update

Responses

Status Code: 200 (Ok) - This status code from the camera indicates that the user has been successfully updated.

Response Body:

```
{
  "success": true,
  "message": "User updated successfully"
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The below API will be received in response to the request from the camera.

```
{
  "success": false,
  "message": "Firmware flashing is in progress"
}
```

Error Handling

This section explains how the update user API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.

Remove User API

This section describes the Remove User API, which allows administrators to delete existing user accounts on the platform. This API provides the functionality to remove user credentials and associated roles, ensuring that unauthorized or no longer required users cannot access the platform. The deletion can include removing the user's associated permissions for interacting with cameras and other platform resources.

Note: Access to the Remove User API is restricted to admin users only.

Base URL

`https://<ipaddress-FastCGI server>/api/v1`

Request

Run the following API on the Postman to delete user in the table.

```
Endpoint: `/users/delete`
Method: `POST`

Basic Authentication:
Username: xxx
Password: Test@123

Request Path Parameters (JSON) :
{
  "username": "test@123",
}
```

Example

`https://<ip-address- fastCGI server>/api/v1/users/delete`

Responses

Status Code: 200 (Ok) – This status code from the camera indicates that the user has been deleted successfully.

Response Body:

The below API will be received in response to the request from the host PC.

```
{
  "success": true,
  "message": "User deleted successfully"
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{
  "success": false,
  "message": "Firmware flashing is in progress"
}
```

Error Handling

This section describes how delete user API communicates errors.

- **Error (401 Unauthorized):** If basic authentication fails,

Response Body:

```
{
  "success": false,
  "message": "Unauthorized access"
}
```

- **Error (401 Unauthorized):** If the user is logged in and attempts to use the delete_user API without proper permissions.

```
{
  "status": "failure",
  "message": "You are not logged in admin user."
}
```

- **Error (400 Bad Request):** If the request is made using GET method.
- **Error (500 Internal Server Error):** If user is currently unavailable, attempt to delete the user information,

```
{
  "success": false,
  "message": "Error deleting user"
}
```

Get Http mode API

This section describes the API for checking the operational status of the device's web services. It allows to verify whether the device is reachable via standard (HTTP) or secure (HTTPS) web protocols.

Note: This API is accessible by both admin and user roles, enabling authorized users to view network configuration details as needed.

Base URL

`https://<ipaddress-FastCGI server>/api/v1`

Request

Run the following API on the postman to retrieve the HTTP details for a camera.

Endpoint: ``/network/http``

Method: ``GET``

Basic Authentication:

Username: `xxx`

Password: `Test@123`

Request Body (JSON) :

```
{  
}
```

Example

`https://<ipaddress-FastCGI server>/api/v1/network/http`

Responses

Status Code: 200 (Ok) – This status code from the camera indicates that the http mode information has been retrieved successfully.

```
{  
  "httpEnabled": false,  
  "httpsEnabled": true  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how the get http mode API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Set Http mode API

This section describes the API for modifying the operational status of the device's web management services. This endpoint to harden device security by enabling or disabling specific web protocols.

Note: This API is accessible by both admin and user roles, enabling authorized users to view network configuration details as needed.

Base URL

`https://<ipaddress-FastCGI server>/api/v1`

Request

Run the following API on the host PC to set the http mode services of a camera.

```
Endpoint: `/network/http`
Method: `POST`

Basic Authentication:
Username: xxx
Password: Test@123

Request JSON Parameters:
{
  "httpEnabled": true,
  "httpsEnabled": true
}
```

Example

`https://<ip-address- fastCGI server>/api/v1/network/http`

Responses

Status Code: 200 (Ok) – This status code from the camera indicates the http mode sets successfully and server was restarted.

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how the set http mode API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Get IPv4 Config API

This section describes the IPv4 API, which retrieves the current IPv4 network configuration of a device, providing details such as DHCP mode, IP address, subnet mask, default gateway, and DNS settings. This API is essential for monitoring and verifying the network settings of devices to ensure they are properly configured for connectivity.

Note: This API is accessible by admin and user roles, enabling authorized users to view network configuration details as needed.

Base URL

`https://<ipaddress-FastCGI server>/api/v1`

Request

Run the following API on the postman to retrieve the ipv4 network details for a camera.

Endpoint: ``/network/ipv4``

Method: ``GET``

Basic Authentication:

Username: `xxx`

Password: `Test@123`

Request Body (JSON):

```
{  
}
```

Example

`https://<ip-address- fastCGI server>/api/v1/network/ipv4`

Responses

Status Code: 200 (Ok) – This status code from the camera indicates that the ipv4 network config details are successfully retrieved.

Response Body:

The below API will be received in response to the request from the host PC.

```
{
  "enabled": true,
  "dhcpEnabled": true,
  "address": "192.168.0.3",
  "netmask": "255.255.255.0",
  "gateway": "192.168.88.1",
  "dns": [
    "192.168.88.1",
    "192.168.88.1"
  ]
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{
  "success": false,
  "message": "Firmware flashing is in progress"
}
```

Error Handling

This section describes how the get ipv4 network config API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Set IPv4 Config API

This section describes the IPv4 API, which sets the current IPv4 network configuration of a device, providing details such as DHCP mode, IP address, subnet mask, default gateway, and DNS settings.

Note:

- This API is accessible by admin and user roles, enabling authorized users to view network configuration details as needed.
- If applying a new IPv4 configuration fails, it now falls back to the network mode that was active before the change — DHCP if it was previously on DHCP, or the previous static configuration if it was previously static.

Base URL

`https://<ipaddress-FastCGI server>/api/v1`

Request

Run the following API on the postman to sets the ipv4 network details for a camera.

```
Endpoint: `/network/ipv4`
Method: `POST`

Basic Authentication:
Username: xxx
Password: Test@123

Request Body (JSON):
{
  "enabled": true,
  "dhcpEnabled": false,
  "address": "192.168.88.101",
  "netmask": "255.255.255.0",
  "gateway": "192.168.88.1",
  "dns": ["8.8.8.8", "1.1.1.1"]
}
```

Example

`https://<ip-address- fastCGI server>/api/v1/network/ipv4`

Responses

Status Code: 200 (Ok) – This status code from the camera indicates that the ipv4 network config details has been successfully updated

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{
  "success": false,
  "message": "Firmware flashing is in progress"
}
```

Error Handling

This section describes how the set ipv4 network config API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Get IPv6 Config API

This section describes the API for retrieving the current IPv6 address details of the device. This is used to verify the device's reachability on IPv6-enabled networks and to identify how the address was assigned.

Note: This API is accessible by admin and user roles, enabling authorized users to view network configuration details as needed.

Base URL

`https://<ipaddress-FastCGI server>/api/v1`

Request

Run the following API on the postman to retrieve the ipv6 network details for a camera.

Endpoint: `/network/ipv6``

Method: `GET``

Basic Authentication:

Username: `xxx`

Password: `Test@123`

Request Body (JSON) :

```
{  
}
```

Example

`https://<ip-address- fastCGI server>/api/v1/network/ipv6`

Responses

Status Code: 200 (Ok) – This status code from the camera indicates that the ipv6 network config details are successfully retrieved.

Response Body:

The below API will be received in response to the request from the host PC.

```
{  
  "address": "fe80::6ff:fe00:1006",  
}
```

```
"source": "linkLocal"  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how the get ipv6 network config API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Get ONVIF Status API

This section describes the API for checking the status of the ONVIF service.

Note: This API is accessible by admin and user roles, enabling authorized users to view network configuration details as needed.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on the postman to retrieve the onvif status of the camera.

Endpoint: ``/network/onvif``

Method: ``GET``

Basic Authentication:

Username: `xxx`

Password: `Test@123`

Request Body (JSON):

```
{  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/network/onvif

Responses

Status Code: 200 (Ok) – This status code from the camera indicates that the onvif status are successfully retrieved.

Response Body:

The below API will be received in response to the request from the host PC.

```
{  
  "enabled": true  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{
  "success": false,
  "message": "Firmware flashing is in progress"
}
```

Error Handling

This section describes how the get onvif status API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Set ONVIF Status API

This section describes the API for enabling or disabling the ONVIF service.

Note: This API is accessible by admin and user roles, enabling authorized users to view network configuration details as needed.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on the postman to sets the onvif status to the camera.

Endpoint: ``/network/onvif``

Method: ``POST``

Basic Authentication:

Username: `xxx`

Password: `Test@123`

Request Body (JSON):

```
{
  "enabled": true
}
```

Example

https://<ip-address- fastCGI server>/api/v1/network/onvif

Responses

Status Code: 200 (Ok) – This status code from the camera indicates that the onvif status has been successfully updated.

Response Body:

The below API will be received in response to the request from the host PC.

```
{
  "enabled": true
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{
  "success": false,
  "message": "Firmware flashing is in progress"
}
```

Error Handling

This section describes how the sets onvif status API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Get RTP Status API

This section describes the API for checking the current **Real-time Transport Protocol (RTP)** streaming mode. This setting determines how video data is encapsulated and transmitted over the network, as well as the security level of the stream

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on the postman to retrieve the rtp status of the camera.

Endpoint: ``/network/rtp``

Method: ``GET``

Basic Authentication:

Username: `xxx`

Password: `Test@123`

Request Body (JSON):

```
{  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/network/rtp

Responses

Status Code: 200 (Ok) – This status code from the camera indicates that the rtp status are successfully retrieved.

Response Body:

The below API will be received in response to the request from the host PC.

```
{  
  "mode": "rtsp"  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how the get rtp status API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Set RTP Status API

This section describes the API for configuring the Real-time Transport Protocol (RTP) delivery method. This setting is critical for defining how the device handles outgoing video streams and whether those streams are encrypted.

Note: The supported mode values are **rtsp**, **rtsp**s, and **disabled**.

Base URL

<https://<ipaddress-FastCGI server>/api/v1>

Request

Run the following API on the postman to sets the rtp status to the camera.

```
Endpoint: `/network/rtp`  
Method: `POST`  
  
Basic Authentication:  
Username: xxx  
Password: Test@123  
  
Request Body (JSON):  
{  
  "mode": "rtsp"  
}
```

Example

<https://<ip-address- fastCGI server>/api/v1/network/rtp>

Responses

Status Code: 200 (Ok) – This status code from the camera indicates that the rtp status has been successfully updated.

Response Body:

The below API will be received in response to the request from the host PC.

```
{  
  "mode": "rtsp"  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{
  "success": false,
  "message": "Firmware flashing is in progress"
}
```

Error Handling

This section describes how the sets rtp status API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Get RTP sessions API

This section describes the API that allows users to retrieve the list of active RTP streaming sessions on the camera. The API returns information about each active session, including the session ID, client IP address, streaming protocol, and session status.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API in Postman to retrieve the active RTP streaming sessions.

```
Endpoint: `/network/rtp/sessions`  
Method: `GET`  
  
Basic Authentication:  
Username: xxx  
Password: Test@123  
  
Request Body (JSON):  
{  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/network/rtp/sessions

Responses

Status Code: 200 (Ok) – This status code indicates that the list of active RTP streaming sessions was retrieved successfully.

Response Body:

The below API will be received in response to the request from the host PC.

```
{  
  "sessions": [  
    {  
      "id": 0,
```

```
    "enabled": true,  
    "clientId": "192.168.0.2",  
    "protocol": "rtsp"  
  },  
  {  
    "id": 1,  
    "enabled": true,  
    "clientId": "192.168.0.2",  
    "protocol": "rtsp"  
  }  
]  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how the Get RTP Sessions API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Terminate RTP sessions API

This section describes the API that allows users to terminate an active RTP streaming session. The API disables the specified RTP session and returns the updated session status.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API in Postman to terminate an active RTP streaming session.

```
Endpoint: `/network/rtp/sessions/{id}/terminate`  
Method: `POST`  
  
Basic Authentication:  
Username: xxx  
Password: Test@123  
  
Request Body (JSON):  
{  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/network/rtp/sessions/0/terminate

Responses

Status Code: 200 (Ok) – This status code indicates that the list of active RTP streaming sessions was retrieved successfully.

Response Body:

This status code indicates that the RTP streaming session was terminated successfully.

```
{  
  "id": 0,  
  "enabled": false  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{
  "success": false,
  "message": "Firmware flashing is in progress"
}
```

Error Handling

This section describes how the Terminate RTP Session API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Get Web UI Status API

This section describes the API for checking the status of the device's web-based Graphical User Interface (GUI). This setting determines whether users can access the management dashboard through a standard web browser.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on the postman to retrieve the webui status of the camera.

Endpoint: ``/network/webui``

Method: ``GET``

Basic Authentication:

Username: `xxx`

Password: `Test@123`

Request Body (JSON):

```
{  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/network/webui

Responses

Status Code: 200 (Ok) – This status code from the camera indicates that the webui status are successfully retrieved.

Response Body:

The below API will be received in response to the request from the host PC.

```
{  
  "enabled": true  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{
  "success": false,
  "message": "Firmware flashing is in progress"
}
```

Error Handling

This section describes how the get web ui status API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Set Web UI Status API

This section describes the API for checking the status of the device's web-based Graphical User Interface (GUI). This setting determines whether users can access the management dashboard through a standard web browser

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on the postman to sets the web UI status to the camera.

Endpoint: `/network/webui``

Method: `POST``

Basic Authentication:

Username: `xxx`

Password: `Test@123`

Request Body (JSON):

```
{  
  "enabled": true  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/network/webui

Responses

Status Code: 200 (Ok) – This status code from the camera indicates that the web ui status has been successfully updated and server was restarted.

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
}
```

```
"message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how the sets web ui status API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Get RSTP Configuration API

This section describes the API that allows users to retrieve the current **Rapid Spanning Tree Protocol (RSTP)** configuration and operational state. The API returns whether RSTP is enabled and its current state on the camera.

Base URL

`https://<ipaddress-FastCGI server>/api/v1`

Request

Run the following API in Postman to retrieve the current RSTP configuration.

Endpoint: `/network/rstp``

Method: `GET``

Basic Authentication:

Username: `xxx`

Password: `Test@123`

Request Body (JSON):

```
{  
}
```

Example

`https://<ip-address- fastCGI server>/api/v1/network/rstp`

Responses

Status Code: 200 (Ok) – This status code indicates that the RSTP configuration was retrieved successfully.

```
{  
  "enabled": true,  
  "state": "forwarding"  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how the Get RSTP Configuration API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Set RSTP Configuration API

This section describes the API that allows users to enable or disable **Rapid Spanning Tree Protocol (RSTP)** on the camera. The API updates the RSTP configuration and returns the current enable status along with the operational state.

Base URL

`https://<ipaddress-FastCGI server>/api/v1`

Request

Run the following API in Postman to enable or disable RSTP.

Endpoint: `/network/rstp``

Method: `POST``

Basic Authentication:

Username: `xxx`

Password: `Test@123`

Request Body (JSON):

```
{
  "enabled": true
}
```

Example

`https://<ip-address- fastCGI server>/api/v1/network/rstp`

Responses

Status Code: 200 (Ok) – This status code indicates that the RSTP configuration was updated successfully.

```
{
  "enabled": true,
  "state": "disabled"
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{
  "success": false,
  "message": "Firmware flashing is in progress"
}
```

Error Handling

This section describes how the Set RSTP Configuration API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Get System Time API

This section describes the API for retrieving the current system time. It provides high-precision UTC time alongside localized time data based on the device's configured offset and daylight-saving settings.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on the postman to retrieve the current internal clock settings of the camera.

```
Endpoint: `/time/clock`  
Method: `GET`  
  
Basic Authentication:  
Username: xxx  
Password: Test@123  
  
Request Body (JSON):  
{  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/time/clock

Responses

Status Code: 200 (Ok) – This status code from the camera indicates that the time settings are successfully retrieved.

Response Body:

The below API will be received in response to the request from the host PC.

```
{  
  "epochSecondsUTC": 1774958400,  
  "timeZoneOffset": "+05:30",  
  "localTimeString": "03/31/2026 05:30:00 PM"
```

```
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how the get system time settings API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Set System time API

This section describes the API for manually setting the device's internal clock. Precise time synchronization is vital for accurate log entry timestamps, scheduled events, and the integrity of recorded video evidence.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on the postman to sets the system time to the camera.

```
Endpoint: `/time/clock`  
Method: `POST`  
  
Basic Authentication:  
Username: xxx  
Password: Test@123  
  
Request Body (JSON):  
{  
  "epochSecondsUTC": 1780930019  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/time/clock

Responses

Status Code: 200 (Ok) – This status code from the camera indicates that the system time updates successfully.

Response Body:

The below API will be received in response to the request from the host PC.

```
{  
  "epochSecondsUTC": 1780930019,  
  "timeZoneOffset": "+5:30",  
  "localTimeString": "06/08/2026 08:16:59 PM"
```

```
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how the sets system time API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Get Time Settings API

This section describes the API for retrieving the device's global time configuration. This includes how the clock is synchronized, which external servers are used, and how the date and time are formatted for the user interface and system logs.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on the postman to retrieve the time source, NTP servers, timezone offset, and local time display format settings.

```
Endpoint: `/time/settings`  
Method: `GET`  
  
Basic Authentication:  
Username: xxx  
Password: Test@123  
  
Request Body (JSON):  
{  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/time/settings

Responses

Status Code: 200 (Ok) – This status code from the camera indicates that the time settings are successfully retrieved.

Response Body:

The below API will be received in response to the request from the host PC.

```
{  
  "source": "manual",  
  "ntpServers": [  
    "192.168.88.180"  
  ]  
}
```

```
],  
  "timeZoneOffset": "+5:30",  
  "useDST": false,  
  "dateFormat": "MM/DD/YYYY",  
  "hourFormat": "12h"  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how the get time settings API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Update time settings API

This section describes the API for configuring the device's time synchronization behavior and regional display preferences. Proper configuration ensures that the device maintains an accurate clock and displays the date and time in a format consistent with local standards.

Note: Changes to the time zone offset, DST setting, date format, or hour format made through this API are reflected in the on-screen timestamp overlay (OSD) shown in the video streams, in addition to the values returned by the Get System Time API.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on the postman to update the time source (manual | ntp_manual | ntp_dhcp) , NTP servers, timezone offset, and local time display format

Note: While choosing the time source as manual, the user needs to set `/time/clock` endpoint with the required epoch time in UTC.

Endpoint: `/time/settings`

Method: `POST`

Basic Authentication:

Username: xxx

Password: Test@123

Request Body (JSON):

```
{
  "source": "manual",
  "ntpServers": [
    "192.168.88.135"
  ],
  "timeZoneOffset": "+5:30",
  "useDST": false,
  "dateFormat": "MM/DD/YYYY",
  "hourFormat": "12h"
}
```

Example

<https://<ip-address- fastCGI server>/api/v1/time/settings>

Responses

Status Code: 200 (Ok) – This status code from the camera indicates that the time settings updates successfully.

Response Body:

The below API will be received in response to the request from the host PC.

```
{
  "source": "manual",
  "ntpServers": [
    "192.168.88.180"
  ],
  "timeZoneOffset": "+5:30",
  "useDST": false,
  "dateFormat": "MM/DD/YYYY",
  "hourFormat": "12h"
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{
  "success": false,
  "message": "Firmware flashing is in progress"
}
```

Error Handling

This section describes how the update time settings API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Download Certificate API

This section describes the Download Certificate API, which allows administrators to download SSL/TLS certificates from the platform securely. This API is useful for obtaining verification, backup, or installation certificates on other devices to maintain secure, encrypted communication across connected systems.

Note: Access to the Download Certificate API is restricted to admin users only.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on the Postman to download the certificate.

Endpoint: `/certificates``

Method: `GET`

Basic Authentication:

Username: `xxx`

Password: `Test@123`

Request Body (JSON Data) :

```
{  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/certificates

Note:

Instead of clicking the “Send” button, click the down arrow on the right side of the “Send” button, and then select “Send and Download.”

Responses

Status Code: 200 (Ok) - The below status code from the camera indicates the download is successful.

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how download certificate API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Download Certificate API

This section describes the Download Certificate API, which allows administrators to download SSL/TLS certificates from the platform securely. This API is useful for obtaining verification, backup, or installation certificates on other devices to maintain secure, encrypted communication across connected systems.

Note: Access to the Download Certificate API is restricted to admin users only.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on the Postman to download the certificate.

Endpoint: `/certificates/pem``

Method: `GET`

Basic Authentication:

Username: `xxx`

Password: `Test@123`

Request Body (JSON Data) :

```
{  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/certificates/pem

Note:

Instead of clicking the “Send” button, click the down arrow on the right side of the “Send” button, and then select “Send and Download.”

Responses

Status Code: 200 (Ok) - The below status code from the camera indicates the download is successful.

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how download certificate API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Upload Certificate API

This section describes the Upload Certificate API, which enables administrators to upload SSL/TLS certificates to the platform securely. This API is essential for setting up or renewing secure connections between cameras, servers, and clients, ensuring data transmission is encrypted and authenticated. Admin can upload certificates in standard formats (e.g., .pem) to configure secure communication channels within the system.

Note: Access to the Upload Certificate API is restricted to admin users only.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on the Postman to upload a certificate.

```
Endpoint: `/certificates`
Method: `POST`

Basic Authentication:
Username: xxx
Password: Test@123

Mandatory Path Parameters:
"filename"

Request Path Parameters (form-data) :
{
File to be given as input (server.pem)
}
```

Example

https://<ip-address- fastCGI server>/api/v1/certificates

Responses

Status Code: 200 (Ok) – This status code from the camera indicates the certificate is uploaded successfully.

Response Body:

```
{  
  "status": "accepted"  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how the upload certificate API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Generate Self-signed Certificate API

This section describes the API for generating and immediately activating a self-signed SSL/TLS certificate. This is particularly useful for establishing encrypted HTTPS or RTSPS communication in environments where a Certificate Authority (CA) is not available or for initial device setup. By defining specific identity parameters, administrators can ensure the device identifies correctly to network scanners and browsers.

Note:

- Access to the Upload Certificate API is restricted to admin users only.
- When a request is sent to this endpoint, the web server restarts internally. As a result, any further requests sent immediately afterward may fail until the web server has finished restarting.

Base URL

`https://<ipaddress-FastCGI server>/api/v1`

Request

Run the following API on the Postman to generate the self-signed certificate.

Endpoint: `/certificates/generate``

Method: `POST``

Basic Authentication:

Username: `xxx`

Password: `Test@123`

Mandatory Path Parameters:

`"filename"`

Request body (JSON Data):

```
{
  "commonName": "suresh",
  "organizationalUnit": "products",
  "organization": "econ",
  "locality": "Chennai",
  "state": "Tamil Nadu",
  "country": "IN",
```

```
"validDays": 365  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/certificates/generate

Responses

Status Code: 200 (Ok) – This status code from the camera indicates the certificate is generated successfully.

Response Body:

```
{  
  "certificatePem": "-----BEGIN CERTIFICATE-----..."  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how the generate self-signed certificate API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Restore Default Certificate API

This section describes the API for reverting the device's SSL/TLS configuration to its factory-default state. This action is typically performed when a custom-uploaded or self-generated certificate has expired, become corrupted, or when the device is being transitioned to a new network environment.

Note:

- Access to the restore certificate API is restricted to admin users only.
- When a request is sent to this endpoint, the web server restarts internally. As a result, any further requests sent immediately afterward may fail until the web server has finished restarting.

Base URL

https://<ipaddress-FastCGI server>/api/v1/

Request

Run the following API on the postman to restore the certificate file in the device.

```
Endpoint: `/certificates/restore-default`  
Method: `POST`  
  
Basic Authentication:  
Username: xxx  
Password: Test@123  
  
Request Path Parameters(form-data):  
{  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/certificates/restore-default

Responses

Status Code: 200 (Ok) - This status code from the camera indicates that the user can restore the default certificate file through this response.

Response Body:

```
{  
  "status": "accepted"  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section explains how the restore the default certificate API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Get Common Video Settings API

This section describes the API for retrieving the current frame rate (FPS) configuration of the camera. The frame rate determines the number of video frames captured and transmitted per second, which directly affects video smoothness, bandwidth usage, and storage requirements.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on Postman to retrieve the common video settings of the camera.

Endpoint: ``/video/common``

Method: ``GET``

Basic Authentication:

Username: `xxx`

Password: `Test@123`

Request Body (JSON) :

```
{  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/video/common

Responses

Status Code: 200 (Ok) – This status code indicates that the common video settings have been successfully retrieved from the camera.

Response Body:

The below API will be received in response to the request from the host PC.

```
{  
  "frameRateFps": 30  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{
  "success": false,
  "message": "Firmware flashing is in progress"
}
```

Error Handling

This section describes how the get common video Settings API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Set Common Video Settings API

This section describes the API for configuring the camera's frame rate (FPS). Adjusting the frame rate allows administrators to balance video smoothness, network bandwidth consumption, and storage requirements based on the application's needs.

The FPS can be set between minimum value **1** to maximum value **30**.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on the host PC to set the common video configuration of a camera.

```
Endpoint: `/video/common`  
Method: `POST`
```

Basic Authentication:

Username: xxx

Password: Test@123

Request Body (JSON) :

```
{  
    "frameRateFps": 30  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/video/common

Responses

Status Code: 200 (Ok) –This status code from the camera indicates that the common video settings were set successfully.

Response Body:

The below API will be received in response to the request from the host PC.

```
{  
    "frameRateFps": 30  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how the set common video settings API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Get Video Streams API

This section describes the API for listing all available video streams supported by the device. This provides a high-level overview of the stream hierarchy, allowing administrators to identify which streams are active and which are available for configuration.

The **ID** parameter specifies the video stream for which the stream setting is being configured or retrieved.

ID	Stream Type
0	Web UI
1	RTSP
2	High
3	Low

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on Postman to retrieve the list of all video streams.

Endpoint: ``/video/streams``

Method: ``GET``

Basic Authentication:

Username: `xxx`

Password: `Test@123`

Request Body (JSON) :

```
{  
}
```

Example

<https://<ip-address- fastCGI server>/api/v1/video/streams>

Responses

Status Code: 200 (Ok) – This status code indicates that the stream list has been successfully retrieved.

Response Body:

The below API will be received in response to the request from the host PC.

```
{
  "streams": [
    {
      "id": 0,
      "enabled": true,
      "label": "stream0"
    },
    {
      "id": 1,
      "enabled": false,
      "label": "stream1"
    },
    {
      "id": 2,
      "enabled": true,
      "label": "stream2"
    },
    {
      "id": 3,
      "enabled": true,
      "label": "stream3"
    }
  ]
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how the get video streams API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Get Specific Video Stream API

This section describes the API for retrieving the detailed configuration of a specific video stream. By using the stream ID, administrators can audit the encoding parameters, bitrate management, recording status, and RTSP pathing for a designated stream.

The **ID** parameter specifies the video stream for which the stream setting is being configured or retrieved

ID	Stream Type
0	Web UI
1	RTSP
2	High
3	Low

Base URL

https://<ipaddress-FastCGI server>/api/v1.

Request

Run the following API to retrieve details about a stream settings with specific ID

```
Endpoint: `/video/streams/{id}`  
Method: `GET`  
Basic Authentication:  
Username: xxx  
Password: Test@123  
Request Body (JSON) :  
{  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/video/streams/2.

Responses

Status Code: 200 (Ok) – This status code from the camera indicates that high resolution settings details have been successfully retrieved.

Response Body:

```
{
  "enabled": true,
  "encoder": {
    "codec": "h265",
    "resolution": "2592x1944",
    "bitrateMode": "CBR",
    "bitrateKbps": 1000,
    "quality": 100,
    "iFrameInterval": 30
  }
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{
  "success": false,
  "message": "Firmware flashing is in progress"
}
```

Error Handling

This section describes how the get the video streams by stream id API handles and reports errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Set Specific Video Stream API

This section describes the API for configuring the detailed encoding, recording, and RTSP transmission parameters for a specific video stream. This endpoint allows administrators to tailor individual streams for high-definition recording, mobile viewing, or web-based live previews.

Controls	Data type	Min value	Max value	Step Value	Default	Accepted Values/ Description
codec	string				"h264"	"h264", or "h265", or "mjpeg"
resolution	string	320x240	2592x1944		"2592x1944"	"2592x1944" "1920x1080" "1280x720" "800x600" "640x480" "320x240"
bitratemode	String				"CBR"	"CBR" or "VBR"
bitrateKbps	Integer	30	5000	1	1000	Unit is Kbps
iFrameInterval	Integer	5	30	1	30	

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on the Postman to set all configuration settings for a camera using the specified stream id.

Endpoint: `/video/streams/{id}`
Method: `POST`
Basic Authentication:

```
Username: xxx
Password: Test@123
Request Body(JSON Data):
{
  "enabled": true,
  "encoder": {
    "codec": "h264",
    "resolution": "1280x720",
    "bitrateMode": "CBR",
    "bitrateKbps": 1000,
    "quality": 100,
    "iFrameInterval": 30
  }
}
```

Example

<https://<ip-address- fastCGI server>/api/v1/video/streams/3>

Responses

Status Code: 200 (Ok) –This status code indicates that the stream settings have been successfully applied.

Response Body:

The below API will be received in response to the request from the Postman.

```
{
  "enabled": true,
  "encoder": {
    "codec": "h264",
    "resolution": "1280x720",
    "bitrateMode": "CBR",
    "bitrateKbps": 1000,
    "quality": 100,
    "iFrameInterval": 30
  }
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{
  "success": false,
  "message": "Firmware flashing is in progress"
}
```

Note:

1. Please make sure the following is used in the payload for the corresponding streams id, setting with other values will return error code 400.

Stream ID	Valid Data
0	codec is always MJPEG bitrateMode is always CBR bitrateKbps is always 1000 iFrameInterval is always 30
2,3	codec cannot be MJPEG

2. As per the SoC documentation, the maximum encoding capacity of encoder (H264 or H265, MJPEG) is 10M @ 30fps. Hence, it is not recommended to have two streams set at max resolution (2592x1944 = 5M) with same codec format. (h264 and h265 share the same hardware codec engine).
3. When bitrateMode is "CBR", the encoder QP is locked and only bitrateKbps controls the encoding rate. When bitrateMode is "VBR", the quality field adjusts the encoder QP and bitrateKbps instead acts as the maximum bitrate cap.

Error Handling

This section explains how the set stream settings API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.

- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Get Image Settings API

This section describes the API for retrieving the current imaging and sensor parameters of the camera. This includes basic adjustments like brightness and contrast, as well as advanced sensor controls like Wide Dynamic Range (WDR), exposure modes, and orientation flips.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on Postman to retrieve the current image settings of the camera.

Endpoint: ``/image/settings``

Method: ``GET``

Basic Authentication:

Username: `xxx`

Password: `Test@123`

Request body (JSON Data) :

```
{  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/image/settings

Responses

Status Code: 200 (Ok) – This status code indicates that the imaging parameters have been successfully retrieved from the device.

Response Body:

The below API will be received in response to the request from the host PC.

```
{  
  "brightness": 65,  
  "contrast": 65,  
  "saturation": 65,
```

```
"sharpness": 10,  
"horizontalFlip": 0,  
"verticalFlip": 0,  
"whiteBalanceMode": 0,  
"manualWhiteBalance": 5000,  
"mannualGain": 0,  
"dayNightMode": "day",  
"wdr": 0,  
"dayNightThreshold": 50  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how get image settings API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Set Image Settings API

This section describes the API for configuring the camera's internal imaging and sensor parameters. This allows administrators to optimize image quality for specific environmental conditions, adjust for mounting orientation, and manage exposure behaviour for varying light levels.

Note: Setting the gain to a very high value may cause false positives in motion detection.

Controls	Data type	Min value	Max value	Step Value	Default	Accepted Values/ Description
brightness	integer	0	100	1	50	
contrast	integer	0	100	1	50	
sharpness	integer	0	100	1	50	
saturation	Integer	0	100	1	50	
white balance	Integer	0	2		0	0 – Auto WB 2 – Manual WB
manual white balance	Integer	2000	8000	1	5000	
horizontal flip	Integer	0	1	0	0	0– Horizontal Flip Off 1-Horizontal Flip On
vertical flip	Integer	0	1	0	0	0–Vertical Flip Off 1-Vertical Flip On
manual gain	Integer	0	100	0	0	

wdr	integer	0	1		0	0-WDR Off 1-WDR On
daynight threshold	Integer	0	100	0	50	
daynight mode	String				"auto"	"auto" "day" "night"

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on Postman to sets the image settings to the camera.

```
Endpoint: `/image/settings`
Method: `POST`

Basic Authentication:
Username: xxx
Password: Test@123

Request body(JSON Data):
{
  "brightness": 50,
  "contrast": 50,
  "saturation": 50,
  "sharpness": 100,
  "horizontalFlip": 0,
  "verticalFlip": 0,
  "whiteBalanceMode": 2,
  "manualWhiteBalance": 5000,
  "manualGain": 0,
```

```
"dayNightMode": "day",  
"wdr": 0,  
"dayNightThreshold": 50  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/image/settings

Responses

Status Code: 200 (Ok) – This status code indicates that the imaging parameters have been sets successfully from the device.

Response Body:

The below API will be received in response to the request from the host PC.

```
{  
  "brightness": 50,  
  "contrast": 50,  
  "saturation": 50,  
  "sharpness": 100,  
  "horizontalFlip": 0,  
  "verticalFlip": 0,  
  "whiteBalanceMode": 2,  
  "manualWhiteBalance": 5000,  
  "manualGain": 0,  
  "dayNightMode": "day",  
  "wdr": 0,  
  "dayNightThreshold": 50  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Note: The WDR change will take effect after the device is rebooted.

Error Handling

This section describes how set image settings API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Restore Image Defaults API

This section describes allows users to reset the image configuration. In the Camera Configuration tab restores all camera controls to their default values.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on the host PC to revert all picture and sensor adjustments to their default states.

```
Endpoint: `/image/settings/restore-default`  
Method: `POST`  
  
Basic Authentication:  
Username: xxx  
Password: Test@123  
  
Request Path Parameters (JSON) :  
{  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/image/settings/restore-default

Responses

Status Code: 200 (Ok) – This status code from the camera indicates that the current camera configuration to the camera's memory reset successfully.

Response Body:

The below API will be received in response to the request from the host PC.

```
{  
  "brightness": 50,  
  "contrast": 50,  
  "saturation": 50,  
  "sharpness": 50,
```

```
"horizontalFlip": 0,  
"verticalFlip": 0,  
"whiteBalanceMode": 0,  
"manualWhiteBalance": 5000,  
"manualGain": 0,  
"dayNightMode": "day",  
  "wdr": 0,  
  "dayNightThreshold": 50  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how reset image settings API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Get ROI Exposure

This section describes the API that allows users to retrieve the current ROI (Region of Interest) exposure settings configured on the camera. The API returns the configured ROI exposure coordinates and dimensions, enabling users to verify the active exposure region.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on the Postman to update the roi-exposure.

```
Endpoint: `image/roi-exposure`  
Method: `GET`  
  
Basic Authentication:  
Username: xxx  
Password: Test@123  
  
Request Path Parameters (JSON) :  
{  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/image/roi-exposure

Note:

you can give the value for ROI Exposure as 320x240.

Responses

Status Code: 200 (Ok) – This status code indicates that the ROI exposure settings were retrieved successfully.

Response Body:

```
{  
  "xStart": 50,  
  "yStart": 50,
```

```
"width": 50,  
"height": 50  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how the Get ROI Exposure API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Set ROI Exposure

This section describes the API allows users to configure the ROI exposure settings. You can specify the ROI exposure value as **320x240**, enabling precise control over the camera's exposure region for better image quality and focus in critical areas.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on the Postman to update the roi-exposure.

```
Endpoint: `image/roi-exposure`  
Method: `POST`  
  
Basic Authentication:  
Username: xxx  
Password: Test@123  
  
Request Path Parameters (JSON) :  
{  
  "xStart": 50,  
  "yStart": 50,  
  "width": 50,  
  "height": 50  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/image/roi-exposure

Note:

you can give the value for ROI Exposure as 320x240

Responses

Status Code: 200 (Ok) – This status code from the camera indicates that ROI values update successfully.

Response Body:

```
{  
  "xStart": 50,  
  "yStart": 50,  
  "width": 50,  
  "height": 50  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Note: The maximum ROI exposure area is 320 x 240.

- xStart and width must be within the range 0 - 320.
- yStart and height must be within the range 0 - 240.
- xStart + width must not exceed 320.
- yStart + height must not exceed 240.

Error Handling

This section describes how the set ROI Exposure API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Get Image Frame API

This section describes the API for retrieving a single JPEG frame from a specific video stream. This endpoint supports both real-time "live" snapshots and the retrieval of historical frames from local SD card storage, depending on whether a timestamp is provided in the request.

The **ID** parameter specifies the video stream for which the stream setting is being configured or retrieved.

ID	Stream Type
0	Web UI
1	RTSP
2	High
3	Low

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API to extract an image frame. The frame resolution will match the current configuration of the target stream.

```
Endpoint (current Frame): `/image/frame/streams/{id}`  
Endpoint(timestamp): `/image/frame/streams/{id}/{timestamp}`  
Method: `GET`  
  
Basic Authentication:  
Username: xxx  
Password: Test@123
```

Example

To get the current frame:

`https://<ip-address- fastCGI server>/api/v1/image/frame/streams/{id}`

To get the recorded frame:

`https://<ip-address- fastCGI server>/api/v1/image/frame/streams/{id}/{timestamp}`

Responses

Status Code: 200 (Ok) – The camera successfully processed the request and returns the image data directly.

Response Body:

The below API will be received in response to the request from the host PC.

JPEG image at the resolution configured for the stream

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{
  "success": false,
  "message": "Firmware flashing is in progress"
}
```

Note:

1. Getting current frame (without specifying timestamp) is supported only for stream id 0 (WebUI stream)
2. Getting recorded frame (specifying timestamp) is supported only for stream id 2 and 3 (Video recording)
3. Getting frame from stream id 1 (RTSP stream) is not supported.

Error Handling

This section describes how the get frame API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Get Overlay API

This section describes the API for retrieving the current configuration of the on-screen timestamp in the streaming.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on Postman to check if the timestamp is currently visible on the video streams and where it is positioned.

```
Endpoint: `/overlay`  
Method: `GET`  
  
Basic Authentication:  
Username: xxx  
Password: Test@123  
Request Body (JSON Data):  
{  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/overlay

Responses

Status Code: 200 (Ok) – This status code indicates that the overlay settings have been successfully retrieved.

Response Body:

The below API will be received in response to the request from the host PC.

```
{  
  "timestampEnabled": true,  
  "position": "bottom-right"  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how the get overlay API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Set Overlay API

This section describes the API for configuring the on-screen timestamp display.

Note: Valid values for the position parameter are **top-left**, **top-right**, **bottom-left**, and **bottom-right**.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on the host PC to sets the timestamp position in the streaming.

```
Endpoint: `/overlay`  
Method: `POST`  
  
Basic Authentication:  
Username: xxx  
Password: Test@123  
  
Request Body (JSON):  
{  
  "timestampEnabled": true,  
  "position": "bottom-left"  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/overlay

Responses

Status Code: 200 (Ok) –This status code indicates that the timestamp has been successfully set and applied in the streaming.

Response Body:

The below API will be received in response to the request from the host PC.

```
{
```

```
"timestampEnabled": true,  
"position": "bottom-left"  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how the set overlay API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Get Motion Configuration API

This section describes the API for retrieving the device's motion detection parameters.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on Postman to retrieve the current motion config values.

```
Endpoint: `/motion/config`  
Method: `GET`  
  
Basic Authentication:  
Username: xxx  
Password: Test@123  
  
Request Body (JSON Data):  
{  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/motion/config

Responses

Status Code: 200 (Ok) – This status code indicates that the motion detection configuration has been successfully retrieved.

Response Body:

The below API will be received in response to the request from the host PC.

```
{  
  "enabled": true,  
  "sensitivity": 85,  
  "motionOnDelay": 1,  
  "motionOffDelay": 1,  
  "motionObjectSize": 5,  
  "maskData": ["0x00", "0x00", "0x01", ....., "0x01"]  
}
```

```
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how the get motion data API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Set Motion Configuration API

This section describes the API for configuring the device's motion data.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on the host PC to sets the motion config data.

```
Endpoint: `/motion/config`
Method: `POST`

Basic Authentication:
Username: xxx
Password: Test@123

Request Body (JSON):
{
  "enabled": true,
  "sensitivity": 45,
  "motionOnDelay": 1,
  "motionOffDelay": 1,
  "motionObjectSize": 5,
  "maskData": ["0x01", "0x01", "0x01", ....., "0x01"]
}
```

Example

https://<ip-address- fastCGI server>/api/v1/motion/config

Responses

Status Code: 200 (Ok) –This status code indicates that the motion config has been successfully set and applied to the camera.

Response Body:

The below API will be received in response to the request from the host PC.

```
{
  "enabled": true,
```

```
"sensitivity": 45,  
"motionOnDelay": 1,  
"motionOffDelay": 1,  
"motionObjectSize": 5,  
"maskData": ["0x01", "0x01", "0x01", ....., "0x01"]  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how the set motion config API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Get Motion Events API

This section describes the API for retrieving the historical log of detected motion events.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on Postman to fetch the list of completed motion events currently stored on the device.

Endpoint: ``/motion/events``

Method: ``GET``

Basic Authentication:

Username: `xxx`

Password: `Test@123`

Request Body (JSON Data):

```
{  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/motion/events

Responses

Status Code: 200 (Ok) – This status code indicates that the motion event list has been successfully retrieved.

Response Body:

The below API will be received in response to the request from the host PC.

```
{  
  "events": [  
    { "motionTransitionId": 40, "timestampUTC":  
1710000500, "durationSeconds": 12 },  
    { "motionTransitionId": 42, "timestampUTC":  
1710000700, "durationSeconds": 5 },  
    { "motionTransitionId": 44, "timestampUTC":
```

```
1710001000, "durationSeconds": 90 }  
],  
"hasMore": false  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Note:

- This endpoint also supports following additional (optional) query parameters for limiting & filtering the motion event list:
 - **sinceld**: filters motion events with Id greater than this. Takes priority over **startTimestampUTC**
 - **startTimestampUTC**: filters motion event with timestamp at or after this. If omitted and **sinceld** not given, starts from oldest timestamp
 - **endTimestampUTC**: filters motion event with timestamp at or before this. If omitted, includes events up to present
 - **limit**: limits max events to be returned. Response includes **hashMore: true** if filtered set was truncated.

Error Handling

This section describes how the get motion event data API communicates errors.

- **401 Unauthorized**: Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Clear Motion Events API

This section describes the API for purging the device's motion event history. This is an administrative action used to reset the activity log, free up space in the event buffer, or prepare the device for a new monitoring period.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on the host PC to permanently delete all recorded motion event metadata from the device.

Endpoint: `/motion/events/clear``

Method: `POST``

Basic Authentication:

Username: `xxx`

Password: `Test@123`

Request Body (JSON) :

```
{  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/motion/events/clear

Responses

Status Code: 200 (Ok) – This status code indicates that the motion event data has been successfully emptied.

Response Body:

The below API will be received in response to the request from the host PC.

```
{  
  "status": "accepted"  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how the clear motion API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Get Motion Status API

This section describes the API for polling the real-time activity state of the camera.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on Postman to determine if there is active movement in the camera's field of view at this exact moment.

Endpoint: ``/motion/status``

Method: ``GET``

Basic Authentication:

Username: `xxx`

Password: `Test@123`

Request Body (JSON Data):

```
{  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/motion/status

Responses

Status Code: 200 (Ok) – This status code indicates that the live motion status was successfully from the camera device.

Response Body:

The below API will be received in response to the request from the host PC.

```
{  
  "motion": true,  
  "timestampUTC": 1781011355,  
  "motionTransitionId": 0  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how the get motion status data API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Generate Video Clip API

This section describes the API for initiating the extraction of a video clip from recorded footage stored on the device's SD card.

Note:

- Only Stream IDs 2 and 3 are supported.
- Video clips can be generated with durations ranging from **5 to 60 seconds**.

Base URL

`https://<ipaddress-FastCGI server>/api/v1`

Request

Run the following API on the host PC to trigger a clip generation job for a specific stream and timeframe.

Endpoint: ``/clips/streams/{id}/generate``

Method: ``POST``

Basic Authentication:

Username: `xxx`

Password: `Test@123`

Request Body (JSON Data):

```
{
  "timestampUTC": 1710000000,
  "durationSeconds": 15
}
```

Example

`https://<ip-address- fastCGI server>/api/v1/clips/streams/{id}/generate`

Responses

Status Code: 200 (Ok) – The camera has successfully verified the request and the availability of the footage. A unique job identifier is returned to track the generation progress.

Response Body:

The below API will be received in response to the request from the host PC.

```
{  
  "status": "accepted",  
  "clipId": "s3_1781163978_1781163992"  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how the clip generate API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Get Clip Status API

This section describes the API for tracking the progress of a video clip generation.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API to check the lifecycle stage and completion percentage of a specific clip.

Endpoint: `/clips/{clip_id}/status``

Method: `GET``

Basic Authentication:

Username: `xxx`

Password: `Test@123`

Request Body (JSON Data):

```
{  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/clips/{clip_id}/status

Responses

Status Code: 200 (Ok) – The status request was successful. The response body indicates the current phase of the clip job.

Response Body:

The below API will be received in response to the request from the host PC.

```
{  
  "status": "completed",  
  "clipId": "s3_1781163978_1781163992"  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how the progress clip status API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Download Video Clip API

This section describes the API for retrieving the final rendered video file once a clip generation job has reached the completed status.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API to download the MP4 file associated with a validated clipId.

Endpoint: ``/clips/{clip_id}/download``

Method: ``GET``

Basic Authentication:

Username: `xxx`

Password: `Test@123`

Request Body (JSON Data):

```
{  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/clips/{clip_id}/download

Note:

Instead of clicking the “Send” button, click the down arrow on the right side of the “Send” button, and then select “Send and Download.”

Responses

Status Code: 200 (Ok) – The request was successful, and the file transfer has begun.

Response Body:

The below API will be received in response to the request from the host PC.

```
{  
  mp4 video data  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how the download clip API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Get SD Card Status API

This section describes the API for retrieving the current operational state of the SD card.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on Postman to determine the status of the sd storage.

Endpoint: ``/storage/sd/status``

Method: ``GET``

Basic Authentication:

Username: `xxx`

Password: `Test@123`

Request Body (JSON) :

```
{  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/storage/sd/status

Responses

Status Code: 200 (Ok) – This status code indicates that the storage management service has successfully polled the hardware state.

Response Body:

The below API will be received in response to the request from the host PC.

```
{  
  "status": "mounted"  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how the get sd card status API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Get SD Card Usage API

This section describes the API for retrieving the current storage consumption of the SD card.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on Postman to retrieve the current utilization of the local storage media.

Endpoint: `/storage/sd/usage``

Method: ``GET``

Basic Authentication:

Username: `xxx`

Password: `Test@123`

Request Body (JSON):

```
{  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/storage/sd/usage

Responses

Status Code: 200 (Ok) – This status code indicates that the storage filesystem was successfully queried for current capacity and usage statistics.

Response Body:

The below API will be received in response to the request from the host PC.

```
{  
  "usedBytes": 16114515968,  
  "freeBytes": 14209253376,  
  "totalBytes": 30323769344  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how the Get sd card usage API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Format SD Card API

This section describes the API for initiating a low-level format of the SD card. This process erases all data—including recorded video, motion event logs, and saved clips—and re-initializes the filesystem to ensure optimal performance and storage reliability.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on the host PC to trigger the formatting procedure.

Endpoint: ``/storage/sd/format``

Method: ``POST``

Basic Authentication:

Username: `xxx`

Password: `Test@123`

Request Body (JSON):

```
{  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/storage/sd/format

Responses

Status Code: 200 (Ok) – The camera has successfully received the command and has initiated the formatting process in the background.

Response Body:

The below API will be received in response to the request from the host PC.

```
{  
  "status": "accepted"  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how the format sd card API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Recover SD Card API

This section describes the API for initiating a filesystem consistency check and repair process (like fsck). This action is used to resolve minor data corruption, index errors, or mount failures that can occur after an improper power loss or unexpected card removal.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on the host PC to attempt a repair of the SD card's filesystem without erasing existing data.

Endpoint: ``/storage/sd/recover``

Method: ``POST``

Basic Authentication:

Username: `xxx`

Password: `Test@123`

Request Body (JSON) :

```
{  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/storage/sd/recover

Responses

Status Code: 200 (Ok) – The camera has successfully received the recovery command and has moved the storage subsystem into recovery mode.

Response Body:

The below API will be received in response to the request from the host PC.

```
{  
  "status": "accepted"  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how the recover sd card API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Mount SD Card API

This section describes the API for mounting the SD card storage device.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on Postman to mount the SD card storage device.

Endpoint: ``/storage/sd/mount``

Method: ``POST``

Basic Authentication:

Username: `xxx`

Password: `Test@123`

Request Body (JSON):

```
{  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/storage/sd/mount

Responses

Status Code: 200 (Ok) – This status code indicates that the SD card was successfully mounted and is ready for use.

Response Body:

The below API will be received in response to the request from the host PC.

```
{  
  "status": "accepted"  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how the mount SD card API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Unmount SD Card API

This section describes the API for unmounting the SD card storage device.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on Postman to unmount the SD card storage device.

Endpoint: `/storage/sd/unmount``

Method: `POST``

Basic Authentication:

Username: `xxx`

Password: `Test@123`

Request Body (JSON) :

```
{  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/storage/sd/unmount

Responses

Status Code: 200 (Ok) – This status code indicates that the SD card was successfully unmounted.

Response Body:

The below API will be received in response to the request from the host PC.

```
{  
  "status": "accepted"  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how the unmount SD card API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Get Firmware Version API

This section describes the API for retrieving the firmware version of the device, which provides users with information about the current software version installed on the camera.

Base URL

`https://<ipaddress-FastCGI server>/api/v1`

Request

Run the following API on the postman to retrieve the Firmware version for a camera.

Endpoint: ``firmware/version``

Method: ``GET``

Basic Authentication:

Username: `xxx`

Password: `Test@123`

Request Path Parameters:

```
{  
  
}
```

Example

`https://<ipaddress-FastCGI server>/api/v1/firmware/version`

Responses

Status Code: 200 (Ok) – This status code from the camera indicates that the firmware version information has been retrieved successfully.

Response Body:

```
{  
  
  "version": "1.13.8"  
  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how the firmware version API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Firmware Upload API

This section describes the API for transferring a signed firmware update package to the device's temporary internal storage. This endpoint handles the data transfer only; the actual installation and flashing process must be triggered by a separate command once the upload is verified.

Note: Access to the Firmware Update API is restricted to admin users only.

Base URL

`https://<ipaddress-FastCGI server>/api/v1`

Request

Run the following API on the Postman to upload the firmware.

```
Endpoint: `firmware/image/upload`  
Method: `POST`  
  
Basic Authentication:  
Username: xxx  
Password: Test@123  
  
Request Body:  
{  
  Content-Type: application/octet-stream  
  Body: Binary firmware payload (Raw bytes)  
}
```

Example

`https://<ip-address- fastCGI server>/api/v1/firmware/image/upload`

Responses

Status Code: 200 (Ok) – This status code from the camera indicates the firmware upload successfully.

Response Body:

```
{  
  "status": "accepted"
```

```
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how the firmware upload API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Firmware Update API

This section describes the API for updating the firmware on the device, allowing users to install the latest software version to enhance performance, security, and feature set on the camera.

Note: Access to the Firmware Update API is restricted to admin users only.

Base URL

`https://<ipaddress-FastCGI server>/api/v1`

Request

Run the following API on the Postman to update the firmware.

```
Endpoint: `firmware/update/start`  
Method: `POST`  
  
Basic Authentication:  
Username: xxx  
Password: Test@123  
  
Request Body (JSON Data):  
{  
}
```

Example

`https://<ip-address- fastCGI server>/api/v1/firmware/update/start`

Responses

Status Code: 200 (Ok) – This status code from the camera indicates the firmware update successfully and device was rebooted.

Response Body:

```
{  
  "status": "accepted"  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how the firmware update API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Get Firmware Update Status API

This section describes the API for monitoring the lifecycle of a firmware update.

Base URL

https://<ipaddress-FastCGI server>/api/v1

Request

Run the following API on Postman to check the current state of the firmware subsystem.

Endpoint: ``/firmware/update-status``

Method: ``GET``

Basic Authentication:

Username: `xxx`

Password: `Test@123`

Request Body (JSON) :

```
{  
}
```

Example

https://<ip-address- fastCGI server>/api/v1/firmware/update-status

Responses

Status Code: 200 (Ok) – The status query was successful. The response body details both the active state and the historical result of the previous update.

Response Body:

The below API will be received in response to the request from the host PC.

```
{  
  "status": "success",  
  "lastFirmwareVersion": "0.141.3"  
}
```

Status Code: 503 (Service Not Available) - If the client has already started a firmware update, this status code indicates that the flashing process is currently underway.

Response Body:

The following response will be received.

```
{  
  "success": false,  
  "message": "Firmware flashing is in progress"  
}
```

Error Handling

This section describes how the Get firmware update status API communicates errors.

- **401 Unauthorized:** Returned when authentication fails due to invalid, missing, or incorrect credentials.
- **400 Bad Request** – The request sent by the client is invalid, malformed, contains missing required parameters, or includes incorrect data.
- **500 Internal Server Error** – An unexpected error occurred on the server while processing the request.
- **403 Forbidden** – The authenticated user is not authorized to access the requested resource.

Related Documents

For getting started with RouteCAM_P_CU514_CHLW_BX_H03R1:

1. For detailed information on getting started with RouteCAM_P_CU514_CHLW_BX_H03R1, please refer to the **KAVRO_Getting_Started_Guide.pdf**.
2. This guide provides detailed instructions on the initial setup process including camera configuration, firmware updates, and ensuring proper streaming functionality. Following these instructions will ensure that the camera is set up correctly and ready for operation

To know more about Camera Web UI:

1. For detailed information on configuring and managing the Camera Web User Interface (UI), please refer to the **KAVRO_Camera_Web_UI_User_Manual.pdf**
2. This document provides a detailed overview how to access and utilize the web-based interface to configure camera settings, manage network connections, adjust image parameters, and control other features directly through your browser. It is essential for setting up the camera and understanding the functionality offered via the web interface.

To know more about RouteCAM_P_CU514_CHLW_BX_H03R1 ONVIF:

1. To understand the ONVIF protocol implementation for the RouteCAM_P_CU514_CHLW_BX_H03R1 camera, please refer to the **KAVRO_ONVIF_User_Manual.pdf**.
2. This manual provides an overview on discovering the device, implementing the ONVIF calls, and accessing the ONVIF services on the camera using an ONVIF client.

To know more about SD card:

1. For detailed information regarding SD card usage, refer to **KAVRO_SDCard_User_Manual.pdf**.
2. This document covers SD card installation, supported filesystem requirements, card formatting, mounting and unmounting procedures, recording storage behaviour, retention management, and SD card status monitoring. It also includes troubleshooting information related to SD card operation and recording functionality.

To Know more about Clip generation:

1. For detailed instructions on generating and retrieving video clips from recorded footage, refer to **KAVRO_ClipGeneration_User_Manual.pdf**
2. This guide explains the clip generation workflow, including motion event retrieval, clip creation requests, clip status monitoring, and downloading generated video clips

Glossary

API: Application Programming Interface

CGI: Common Gateway Interface

DHCP: Dynamic Host Configuration Protocol

DNS: Domain Name System

HTTP: Hyper Text Transfer Protocol

JSON: JavaScript Object Notation

NTP: Network Time Protocol

ROI: Region Of Interest

Revision History

Rev	Date	Description	Author
1.0	11-Jun-2026	Alpha Release	Camera Team
1.1	29-July-2026	Intermediate Release	Camera Team
1.2	01-Sep-2026	Intermediate Release	Camera Team